AUTOESTUDO · BOOTCAMP QI TECH 2026

Autoestudo — Aula 4 (03/10) · Produção

A Aula 4 é expositiva — menos pauta, mais exposição. Ela trata do que muda quando um software sai da máquina de quem escreveu e passa a rodar em produção, sob carga real e falha real, em três blocos: **enxergar o sistema em produção** (logs, métricas, tracing e health check), **aguentar carga e falha** (balanceamento, autoscaling, timeouts, retry, circuit breaker e idempotência) e **segurança em produção** (configuração, o Top 10 de risco de API, autenticação, autorização e TLS). **Caminho recomendado:** `2h21;` lendo também os complementos em português, mais 52 minutos.

Cada tópico traz por que ele importa na aula, de 1 a 3 materiais (tipo, idioma e duração declarados) e uma pergunta para você chegar no sábado com resposta própria.

BLOCO 1 — ENXERGAR O SISTEMA EM PRODUÇÃO

1 Logs estruturados e correlação por request id

POR QUE IMPORTA quando um erro em produção não pode ser reproduzido, o log é o único registro do que aconteceu. Se cada linha não carrega o mesmo identificador da requisição, juntar as peças vira arqueologia — é por isso que um cabeçalho de correlação (por exemplo, um `X-Request-ID`) propagado por toda a cadeia de log de uma chamada é o que permite filtrar, num sistema com vários serviços, exatamente as linhas de uma única transação.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
XI. Logs — Treat logs as event streams https://12factor.net/logs — Adam Wiggins, The Twelve-Factor App (atualizado em 2017)	DOC OFICIAL (METODOLOGIA) INGLÊS ~5 min	A ideia central: o processo não decide para onde o log vai, só escreve o evento em `stdout`; quem roteia e arquiva é o ambiente de execução — não a aplicação.
Melhorando a observabilidade da sua aplicação — Parte 1: Logs https://dev.to/marcosbelorio/melhorando-a-observabilidade-da-sua-aplicacao-parte-1-logs-jm2 — Marcos Belorio, dev.to (18/03/2025)	ARTIGO PORTUGUÊS ~8 min	A diferença entre escrever o dado dentro da string da mensagem e escrevê-lo como metadado do log — é o que permite filtrar por `request_id` depois, sem parsing de texto.

PERGUNTA PARA LEVAR se um cliente reportar um erro às 15h32, qual dado no seu sistema te permite achar exatamente aquela requisição — e não uma parecida?

2 Métricas: golden signals e RED

POR QUE IMPORTA log mostra o que aconteceu num caso; métrica mostra a tendência de milhares de casos. Antes de instrumentar qualquer coisa, a pergunta é: quais 3 ou 4 números, olhados juntos, já dizem se o sistema está saudável agora? É a pergunta que os *golden signals* — e sua variante RED, pensada para serviços orientados a requisição — respondem.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
Monitoring Distributed Systems https://sre.google/sre-book/monitoring-distributed-systems/ — Rob Ewaschuk, ed. Betsy Beyer, Google SRE Book	DOC OFICIAL (LIVRO) INGLÊS ~10 min (seção dos 4 sinais)	As quatro definições exatas — latência, tráfego, erros, saturação — e a frase-chave: "se você só puder medir quatro métricas, foque nessas quatro".
The RED Method: How to Instrument Your Services https://grafana.com/blog/the-red-method-how-to-instrument-your-services/ — Tom Wilkie, via Grafana Labs (03/08/2018)	ARTIGO (ENGENHARIA) INGLÊS ~8 min	RED recorta os golden signals para o caso de serviço: Rate, Errors, Duration — os três números que todo endpoint HTTP deveria expor da mesma forma.

PERGUNTA PARA LEVAR se você só pudesse olhar para 3 números num domingo à noite para saber se o sistema está bem, quais seriam — e por quê?

Complemento (não soma ao total): **Golden Signals: o que são e qual sua importância**
https://www.anselme.com.br/2025/07/23/golden-signals/
— Thiago Anselme (23/07/2025), artigo em português, ~6 min — os quatro sinais em português, com exemplos de erro que passam por autenticação e timeout.

3 Tracing distribuído e health check: liveness × readiness

POR QUE IMPORTA um trace mostra o caminho de uma requisição entre serviços — onde o tempo foi gasto e onde ela falhou. Já o health check não conta uma história: dá um sim/não. O processo está vivo (*liveness*)? Está pronto para receber tráfego (*readiness*)? São sinais de natureza diferente — um narra, o outro decide — e é a rota de saúde que o orquestrador (o `compose` do desafio, ou um cluster em produção) usa para decidir se manda tráfego para aquela instância.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
Traces https://opentelemetry.io/docs/concepts/signals/traces/ — documentação oficial do OpenTelemetry	DOC OFICIAL INGLÊS ~8 min	A frase que resume o conceito: um trace pode ser pensado como "uma coleção de logs estruturados com contexto, correlação e hierarquia já embutidos" — cada span carrega o mesmo trace id.
Liveness, Readiness, and Startup Probes https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/ — documentação oficial do Kubernetes	DOC OFICIAL INGLÊS ~10 min	A distinção prática: falha de liveness **reinicia** o container; falha de readiness só **tira ele da lista de quem recebe tráfego** — e a doc avisa que liveness mal configurado gera reinício em cascata.

PERGUNTA PARA LEVAR das duas perguntas — "o processo está vivo?" e "o processo está pronto para tráfego?" — qual delas o endpoint de saúde do seu desafio responde hoje?

Complemento (não soma ao total): **Traces na Observabilidade: span, trace_id e rastreamento distribuído**
https://www.opservices.com.br/traces/
— Pedro Tebaldi, OpServices (05/12/2025), artigo em português, ~10 min — o mesmo conceito em português, com o trace_id como elo entre logs, métricas e traces.

BLOCO 2 — AGUENTAR CARGA E FALHA

4 Load balancing e autoscaling horizontal (por que a app precisa ser sem estado)

POR QUE IMPORTA quando uma instância não aguenta mais tráfego sozinha, duas decisões andam juntas: distribuir requisições entre várias cópias (balanceamento de carga) e ajustar quantas cópias existem (autoscaling horizontal). A segunda só funciona se a aplicação não guardar estado na memória de uma instância específica — senão a próxima requisição pode cair numa cópia que não tem aquele estado.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
HTTP Load Balancing https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/ — documentação oficial do NGINX	DOC OFICIAL INGLÊS ~10 min	Os métodos de escolha do servidor (round robin, least connections, least time, hash) e a observação de que round robin é o padrão quando nenhum outro é configurado.
Horizontal Pod Autoscaling https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ — documentação oficial do Kubernetes	DOC OFICIAL INGLÊS ~10 min	O algoritmo é simples: `réplicas desejadas = ceil(réplicas atuais × métrica atual / métrica alvo)` — a autoscaling decide **quantas cópias**, não qual delas atende cada requisição; isso é papel do balanceador.

PERGUNTA PARA LEVAR se sua aplicação guarda alguma coisa na memória de um processo (um carrinho, um contador, uma sessão), o que acontece quando o autoscaling sobe uma segunda cópia?

Complemento (não soma ao total): **Escalabilidade: o que é e como funciona na prática**
https://curitibablog.com.br/entendendo-escalabilidade-na-pratica
— Wallyson Duarte, CuritibaBlog (27/12/2025), artigo em português, ~8 min — escala horizontal e autoscaling em português, com o aviso de que autoscaling não resolve gargalo de banco de dados.

5 Timeouts, retry com backoff e jitter, e circuit breaker

POR QUE IMPORTA um serviço externo fora do ar não pode travar quem o chama para sempre — por isso todo conector tem um timeout: a diferença entre esperar alguns segundos e devolver um erro claro (um 502, por exemplo), ou travar a requisição indefinidamente. Depois do timeout vem a pergunta seguinte: tentar de novo (retry), com que espera entre tentativas (backoff, com jitter para não sincronizar todos os clientes na mesma tentativa) e quando parar de insistir de vez (circuit breaker).

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
Timeouts, retries, and backoff with jitter https://d1.awsstatic.com/builderslibrary/pdfs/timeouts-retries-and-backoff-with-jitter.pdf — Marc Brooker, Amazon Builders' Library (2019)	DOC OFICIAL (ENGENHARIA AWS) INGLÊS ~12 min	O motivo do jitter: se todos os clientes recuam pelo mesmo tempo, eles colidem de novo na mesma hora; jitter espalha as tentativas no tempo. E a frase de alerta: "retries são egoístas" — cada uma pede mais do servidor.
CircuitBreaker https://martinfowler.com/bliki/CircuitBreaker.html — Martin Fowler (06/03/2014)	VERBETE INGLÊS ~8 min	Os três estados — fechado, aberto, semiaberto — e a diferença de propósito: timeout limita **quanto tempo você espera**; circuit breaker decide **se vale a pena nem tentar**.

PERGUNTA PARA LEVAR o conector do seu desafio para o serviço externo tem timeout? E o que o seu código faz quando ele estoura — tenta de novo, devolve erro, ou trava a requisição?

Complemento (não soma ao total): **Circuit Breaker: Guia completo para APIs resilientes**
https://blog.hubdodesenvolvedor.com.br/circuit-breaker-guia-apis-resilientes/
— Gabriella Martins, Hub do Desenvolvedor (08/12/2025), artigo em português, ~7 min — a mesma distinção timeout × circuit breaker em português.

6 Idempotência: a pré-condição do retry

POR QUE IMPORTA retry só é seguro se repetir a mesma operação não duplicar o efeito. Lançar duas vezes a mesma tarifa porque o retry reenviou uma transação que, na verdade, já tinha sido processada do outro lado é o tipo de bug que só aparece em produção, sob falha real de rede — e a idempotência é a garantia que evita isso, antes mesmo de decidir a política de retry.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
Making retries safe with idempotent APIs https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/ — Malcolm Featonby, Amazon Builders' Library	DOC OFICIAL (ENGENHARIA AWS) INGLÊS ~10 min	A definição prática: um identificador único gerado pelo cliente permite que o servidor reconheça uma repetição e devolva a mesma resposta, em vez de processar de novo.
Entendendo a idempotência e resiliência em APIs https://dev.to/asouza/entendendo-a-idempotencia-e-resiliencia-em-apis-23e2 — Alberto Luiz Souza, dev.to (07/10/2024)	ARTIGO PORTUGUÊS ~7 min	O exemplo é justamente um pagamento que dá timeout e é reenviado pelo cliente — e por que comparar só os parâmetros da requisição não basta, sem uma chave de idempotência explícita.

PERGUNTA PARA LEVAR qual operação do seu desafio — criar cliente, lançar transação, cobrar tarifa — seria perigosa de repetir sem querer, e o que te protegeria disso?

BLOCO 3 — SEGURANÇA EM PRODUÇÃO

7 Segredos e configuração (12-factor)

POR QUE IMPORTA toda configuração que muda entre ambientes — senha de banco, endereço de um serviço externo, token de acesso — fica fora do código, em variável de ambiente, nunca em arquivo versionado. É a mesma razão pela qual um arquivo de segredo fica fora do controle de versão e só o arquivo de exemplo (com os nomes das variáveis, sem os valores) é versionado.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
III. Config https://12factor.net/config — Adam Wiggins, The Twelve-Factor App (atualizado em 2017)	DOC OFICIAL (METODOLOGIA) INGLÊS ~5 min	A frase que resume tudo: guardar config como constante no código "é uma violação do doze fatores" — mesmo que o repositório seja privado. Variável de ambiente muda por deploy sem tocar em código.

PERGUNTA PARA LEVAR no seu repositório, onde mora o valor de uma credencial hoje — e o que aconteceria se esse arquivo fosse commitado por engano?

Complemento (não soma ao total, mesmo conteúdo em português): **III. Configuração**
https://12factor.net/pt_br/config
— tradução oficial do The Twelve-Factor App, ~5 min.

8 OWASP API Security Top 10

POR QUE IMPORTA a maior parte dos incidentes de segurança em API não é criptografia quebrada — é autorização quebrada: um endpoint que devolve o registro de outro cliente porque só checou se o token era válido, não se o dono do token tinha direito àquele registro específico. É o item número 1 da lista desde a primeira edição.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
OWASP API Security Top 10 – 2023 https://api-security.owasp.org/editions/2023/en/0x11-t10 — OWASP Foundation	DOC OFICIAL INGLÊS ~10 min	Os 10 itens, na ordem: Broken Object Level Authorization (#1), Broken Authentication, Broken Object Property Level Authorization, Unrestricted Resource Consumption, Broken Function Level Authorization e mais 5 — repare que autorização aparece 3 vezes nos 5 primeiros.

PERGUNTA PARA LEVAR existe alguma rota do seu desafio que devolve um recurso — conta, transação — só checando se o token é válido, sem checar se ele pertence ao dono daquele recurso?

Complemento (não soma ao total, edição anterior em português): **Os dez mais críticos riscos de segurança de API — 2019**
https://api-security.owasp.org/editions/2019/pt-BR/0x00-header
— tradução oficial OWASP (10/03/2020), ~10 min. É a edição de 2019, não a de 2023: a lista mudou de nome e de ordem em alguns itens — leia como reforço conceitual, não como lista atualizada.

9 Autenticação × autorização e TLS

POR QUE IMPORTA autenticar responde quem está falando; autorizar responde o que essa identidade pode fazer; TLS garante que ninguém no meio do caminho lê ou altera o que foi dito. São três problemas diferentes, e confundir os dois primeiros é a causa mais comum por trás dos itens de autorização quebrada do Top 10.

MATERIAL	TIPO · IDIOMA · DURAÇÃO	NO QUE PRESTAR ATENÇÃO
Authentication Cheat Sheet https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html — OWASP Cheat Sheet Series	DOC OFICIAL INGLÊS ~12 min	O dado mais forte da página: MFA sozinho já teria evitado 99,9% dos comprometimentos de conta, segundo a estatística citada. E o aviso: nunca expor login de conta interna/administrativa na mesma interface do usuário final.
O que é um certificado SSL/TLS? https://aws.amazon.com/pt/what-is/ssl-certificate/ — AWS, documentação educacional oficial	DOC OFICIAL PORTUGUÊS ~8 min	O handshake em passos: criptografia assimétrica só para trocar a chave de sessão; depois disso, a conexão inteira passa a usar criptografia simétrica, mais barata computacionalmente.

PERGUNTA PARA LEVAR numa rota do seu desafio, o que garante que quem está entrando é quem diz ser (autenticação) — e o que garante que, mesmo sendo quem diz ser, ele só vê a própria conta (autorização)?

Complemento (não soma ao total): **Autenticação x autorização – qual é a diferença?**
https://www.freecodecamp.org/portuguese/news/autenticacao-x-autorizacao-qual-e-a-diferenca/
— Grant Riordan, trad. Rafael Fontenelle, freeCodeCamp (27/02/2023), artigo em português, ~6 min — a analogia da festa (segurança confere identidade, depois confere o convite) para fixar a diferença.

Caminho recomendado — 2h21

Nesta ordem, com os minutos de cada item: 12factor, Logs (5) · dev.to Belorio, logs estruturados em português (8) · Google SRE Book, os 4 golden signals (10) · Grafana, RED method (8) · OpenTelemetry, Traces (8) · Kubernetes, Liveness/Readiness Probes (10) · NGINX, HTTP Load Balancing (10) · Kubernetes, Horizontal Pod Autoscaling (10) · AWS Builders' Library, Timeouts, retries and backoff with jitter (12) · Fowler, CircuitBreaker (8) · AWS Builders' Library, Making retries safe with idempotent APIs (10) · dev.to Alberto Souza, idempotência em português (7) · 12factor, Config (5) · OWASP API Security Top 10 – 2023 (10) · OWASP, Authentication Cheat Sheet (12) · AWS, O que é um certificado SSL/TLS (8).

Total: 2h21.

Ficam de fora desta soma os sete complementos em português (Anselme, 6 min; OpServices, 10 min; CuritibaBlog, 8 min; Hub do Desenvolvedor, 7 min; 12factor Config PT, 5 min; OWASP API Security 2019 PT, 10 min; freeCodeCamp, 6 min), que somam mais 52 minutos para quem quiser reforço bilíngue ou mais profundidade.

Antes de 03/10

A Aula 4 é teórica e **não é preciso levar computador** — nada precisa estar rodando na sua máquina no sábado. O que vale trazer anotado são as 9 perguntas deste material: elas são discutidas em sala e cada uma reaparece como decisão de arquitetura no seu desafio, entregue em 13/10.